Practical 2: Adversarial Games

If practical 1 versed on the search algorithms on a one-sided variant of chess game, typical chess is possibly one of a kind of adversarial games, in which two opponents confront each other's ability at finding the smartest strategy to check mate each other.

The goal of this practical is to familiarize yourself with strategies to play adversarial games, with the basic search algorithms of AI. To this end, your task will consist of implementing several adversarial game search algorithms to attain specific goals on a chess board.

The Simulator
As for practical 1, you will use the very same simulator you used for the previous practical, consisting of four hierarchical classes: aichess, chess, board, piece. The latter three implement the dynamics of a chess game, which may be run by two players. Please, run the main on the class Chess to this end. Finally, the Aichess class is a capsule of the former three, with the purpose of implementing AI algorithms to analyse and alter the dynamics of the chess game.

A few **notes of interest**

- The Chess class incorporates two instances of the Board class: Board and BoardSim. The idea is that of clean-cut separating a board in which you may simulate your moves, and another one in which you may make your effective moves towards the "target" whenever you are certain of what the correct strategy is.
- In brief, you must use these two boards to simulate your moves, effectively moving the pieces, as a means to synchronize the aichess exploration of the state of the board.
- Unlike in the case of search algorithms, adversarial games, both white and black pieces may move ahead.

The Definition of the State and the Initial State

- The definition of the state ought to include the positions and piece types. Although you may define two independent black/white states, the problem state includes both.
- Team moves in alternation. You will have to implement an algorithm to move whites/blacks as required.

Your Work
As customary in our AI practical, during this game you are provided with a specific starting board configuration (pre-programmed): it contains a black king, in position [0,5], and a Black rook in position [0,7], and two white pieces, the movement of which you will have to program: a king in position [7,5] and a rook in position [7,0]. Your goal will vary from question to question, but keep in mind that, in chess, whites move first. You are instructed to find that state by implementing algorithms described next.

1. Whites start moving. Implement the dynamics of a game in which both, whites and blacks, follow the same Minimax algorithm to try to check-mate each other. Assume that both implement minimax with a depth of 4 moves (2p)
    a. Once implemented, run the same game a few times. How often do whites win?
    b. Provide a justification for that.

2. Now run the same simulations, but varying the depth of the minimax algorithm from 3 to 4 moves both for whites and blacks. Run each possible combination of depths a few times (1.5p)
    a. Plot the percentage of white wins over the total for each depth value.
    b. Is the result symmetric. Why?

3. Implement the alfa-beta pruning for the blacks only, whites still play with minimax (2p).
    a. Using an equal depth of 4, run the simulation three times. Who is the best in each simulation?
    b. Why is that?

4. Both whites and blacks use the same alfa-beta pruning. Run three simulations each while varying the depth with which each team plays (1-5) (1.5p)
    a. Plot the proportion of wins for whites and blacks.
    b. Comment on the result.

5. Implement the expectimax algorithm for whites and blacks (2p). Whites play expectimax, blacks alfa-beta pruning.
    a. Run three simulations each and plot the proportion of wins for whites/blacks.
    b. Who wins the most. Why?

6. The situation generated by confronting a white king and a black king plus a rook each may be considered even (1p)
    a. Is it really the case? Justify your answer.
    b. In your opinion, what makes this situation of particular interest for the study of adversarial games?

Report Guidelines

The report should not exceed 10 pages. Any pages beyond this limit will not be taken into account.

Transparency in AI Use

You must include an AI Use Statement in your report, clearly indicating whether you have used AI at any stage of the project, including code development and report writing.

If you have used AI, clearly specify:

- Which parts of the work were developed with AI, including an estimated percentage of AI contribution.
- Which AI tool(s) were used.
- For what purpose the AI tool(s) were used.

Please note that failure to include this statement will result in a deduction from your grade.

Submission Deadlines & Oral Exams

- **Groups B and F** should submit their work (code + report) by **Wednesday, 28/10, at 23:59**. The respective interviews will take place on **Tuesday, 10/11**, during each group's usual laboratory session.

- **Group A** should submit its work (code + report) by **Thursday, 29/10, at 23:59**. The respective interview will take place on **Wednesday, 11/11**, during the usual laboratory session.

Documentation

The python code provided is documented, and provides a straightforward structure for you to study and analyse. Although you do not have to understand every single detail, you need to build sufficient intuition about the structure of the underlying classes to be able to solve the problems listed previously.

In this practical we view chess as an adversarial game: White and Black alternate moves from an initial position, and each player searches the game tree for moves that lead to check-mate. To facilitate your work, the Aichess class provides:

- getCurrentState(): returns the list of [row, col, piece_id] for all pieces on the board, White and Black.
- getWhiteState(*thisState*) / getBlackState(*thisState*): the white or black pieces of a given state.
- getListNextStatesW(*thisState*) / getListNextStatesB(*thisState*): return the lists of possible states that can be reached from *thisState* for White and Black, respectively. Moves are generated from the simulation board, so boardSim must be updated first, e.g. with newBoardSim or moveSim. Note that moveSim cannot undo captures, so newBoardSim is safer when you backtrack in the search tree.

It is fundamental that you implement the **state** as a list of elements containing the positions and types of pieces you move. Each piece has a numeric identifier you may find in the initialization of the board class. For example, the initial position of this practical is: [[7,0,2],[7,5,6],[0,5,12],[0,7,8]]. This indicates that a white rook is in board position [7,0], the white king on position [7,5], the black king on [0,5] and the black rook on [0,7].